

Preentrega Módulo 2

Arquitectura Multi-Agente para Soporte Técnico

Cecilia Manzano

1. Descripción del proyecto

Se desarrolló en n8n un sistema multi-agente para la gestión automática de tickets de soporte técnico.

La arquitectura implementa el patrón **Manager–Worker**, separando la tarea de clasificación de los tickets de su resolución técnica. El objetivo es evitar un agente monolítico y dividir las responsabilidades entre especialistas con alcances claramente definidos.

El sistema está compuesto por:

- Un **Agente Manager**, encargado de analizar y clasificar cada ticket.
- Un **Worker especializado en Soporte a Usuarios**.
- Un **Worker especializado en Infraestructura**.
- Una vía de escape hacia un **supervisor humano** cuando el ticket no puede clasificarse con suficiente claridad.

2. Arquitectura general

El flujo principal recibe el ticket mediante el chat de n8n.

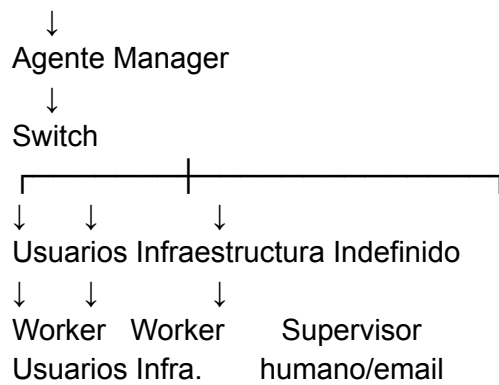
El Agente Manager analiza el lenguaje natural del usuario y devuelve una clasificación estructurada dentro de una taxonomía cerrada:

```
{"tarea":"usuarios"}  
{"tarea":"infraestructura"}  
{"tarea":"indefinido"}
```

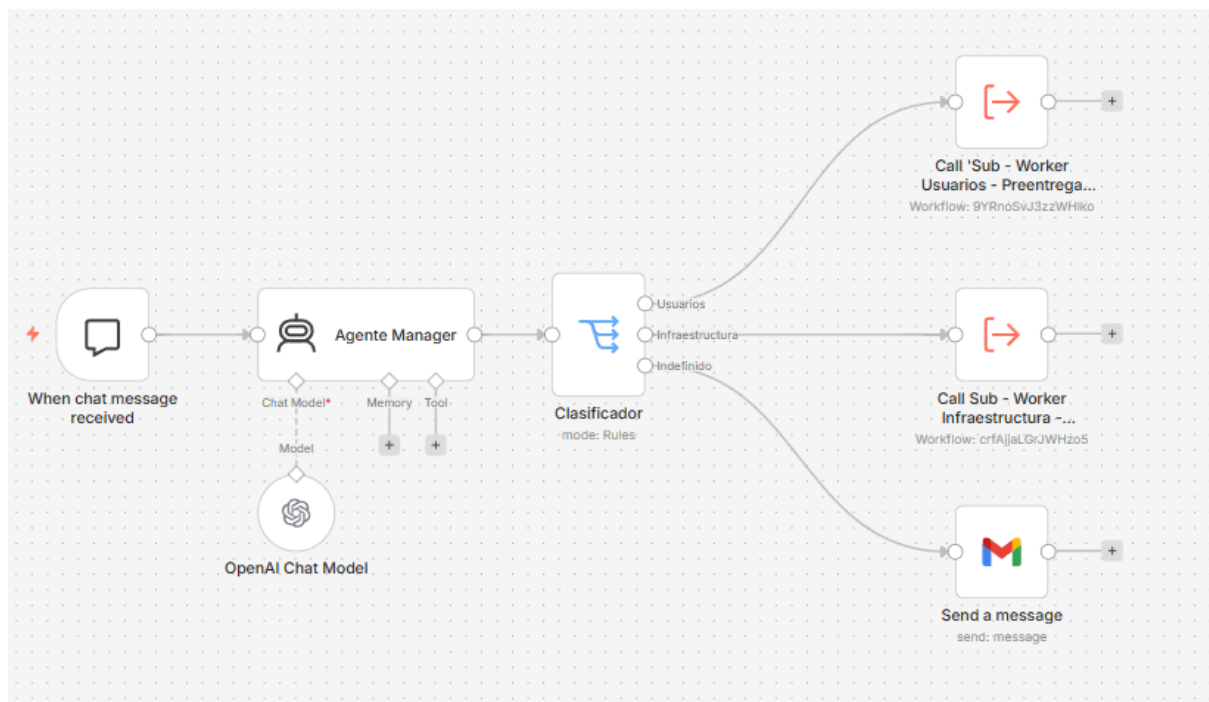
Un nodo **Switch** utiliza este resultado para enrutar el ticket hacia el especialista correspondiente.

La arquitectura general es:

Chat Trigger



Captura — Workflow principal



3. Agente Manager y enrutamiento

El Manager funciona exclusivamente como **agente de triaje**.

No intenta resolver los problemas técnicos. Su responsabilidad consiste en comprender el ticket y determinar qué especialista debe intervenir.

La taxonomía utilizada es:

Usuarios: problemas asociados principalmente a dispositivos finales, computadoras, notebooks, celulares, sistemas operativos, aplicaciones, periféricos y soporte técnico directo al usuario.

Infraestructura: problemas relacionados con redes LAN, Ethernet, switches, routers, access points, cableado, Wi-Fi corporativo, servidores y demás componentes de infraestructura tecnológica.

Indefinido: tickets ambiguos o con información insuficiente para asignarlos responsablemente a uno de los dos especialistas.

La clasificación **indefinido** actúa como vía de escape. Estos casos son derivados mediante Gmail a una persona de soporte técnico para su revisión manual.

4. Worker de Soporte a Usuarios

El Worker de Usuarios fue desarrollado como un **sub-workflow independiente**.

Su alcance incluye principalmente:

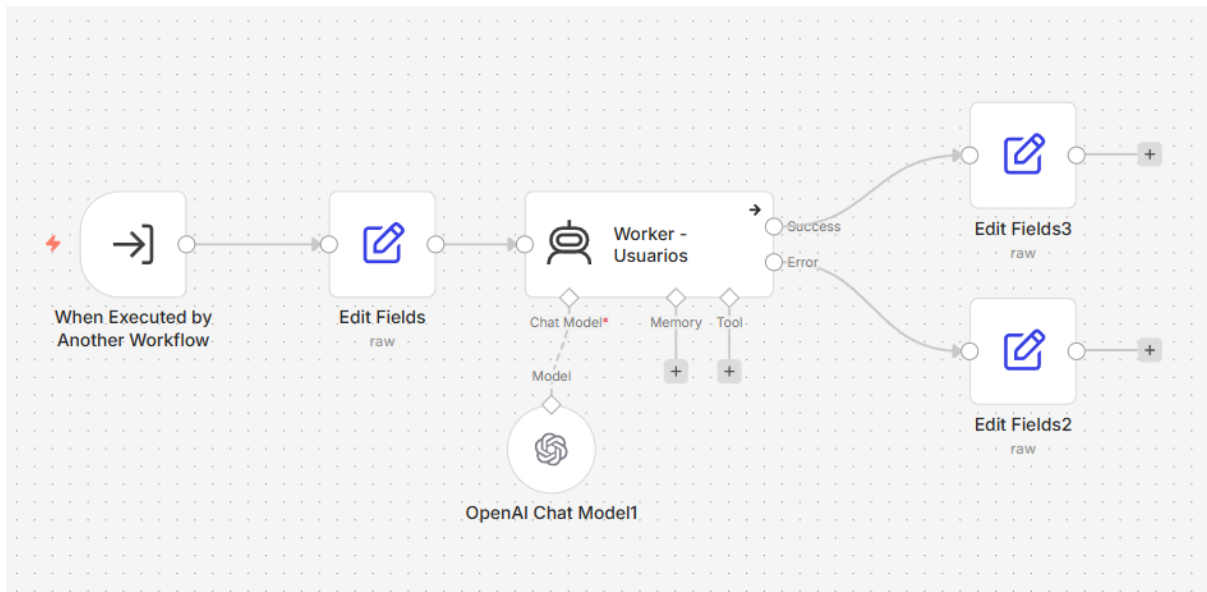
- Computadoras y notebooks.
- Windows y macOS.
- Hardware de dispositivos finales.
- Software y aplicaciones.
- Drivers.
- Periféricos.
- Celulares y tablets.
- Problemas locales de conectividad cuando la falla está limitada al dispositivo del usuario.

El sub-workflow comienza mediante **When Executed by Another Workflow**, permitiendo que sea invocado desde el Manager.

Recibe como entrada únicamente el dato necesario para realizar su tarea:

```
{  
  "ticket": "descripción del problema"  
}
```

Captura — Sub-workflow Worker Usuarios



5. Worker de Infraestructura

El segundo Worker también funciona como un sub-workflow independiente y está especializado en infraestructura IT.

Su alcance comprende:

- Redes LAN y Ethernet.
- Switches.
- Routers.
- Access Points.
- Wi-Fi corporativo.
- VLAN y direccionamiento.
- Cableado estructurado.
- Servidores físicos y virtuales.
- Sistemas operativos de servidores.
- Servicios centrales e infraestructura compartida.

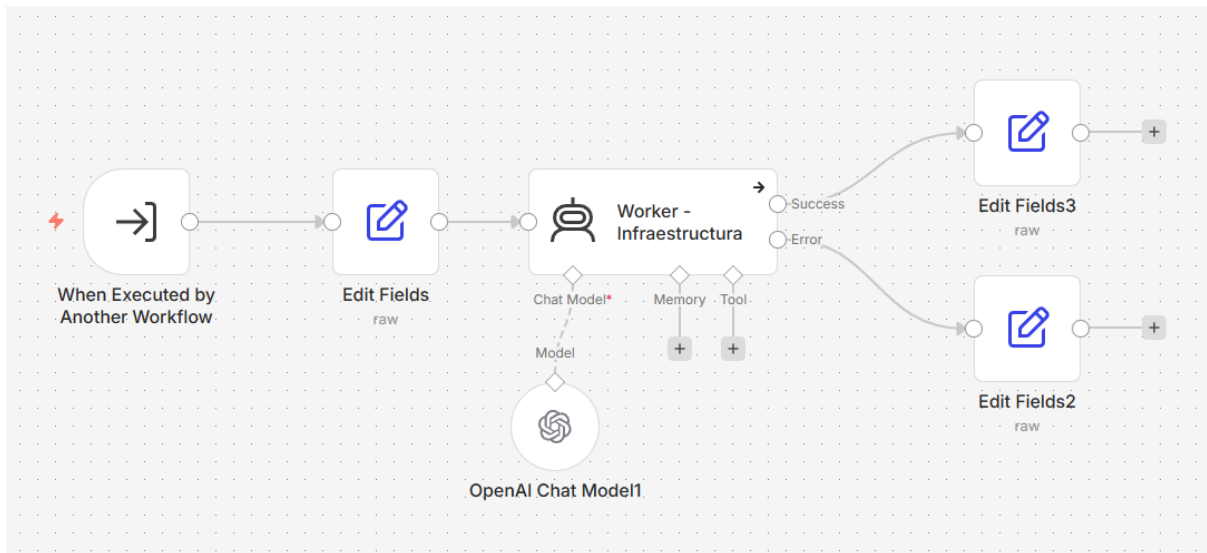
Al igual que el Worker de Usuarios, recibe un contrato de entrada reducido al dato necesario:

```

{
  "ticket": "descripción del problema"
}

```

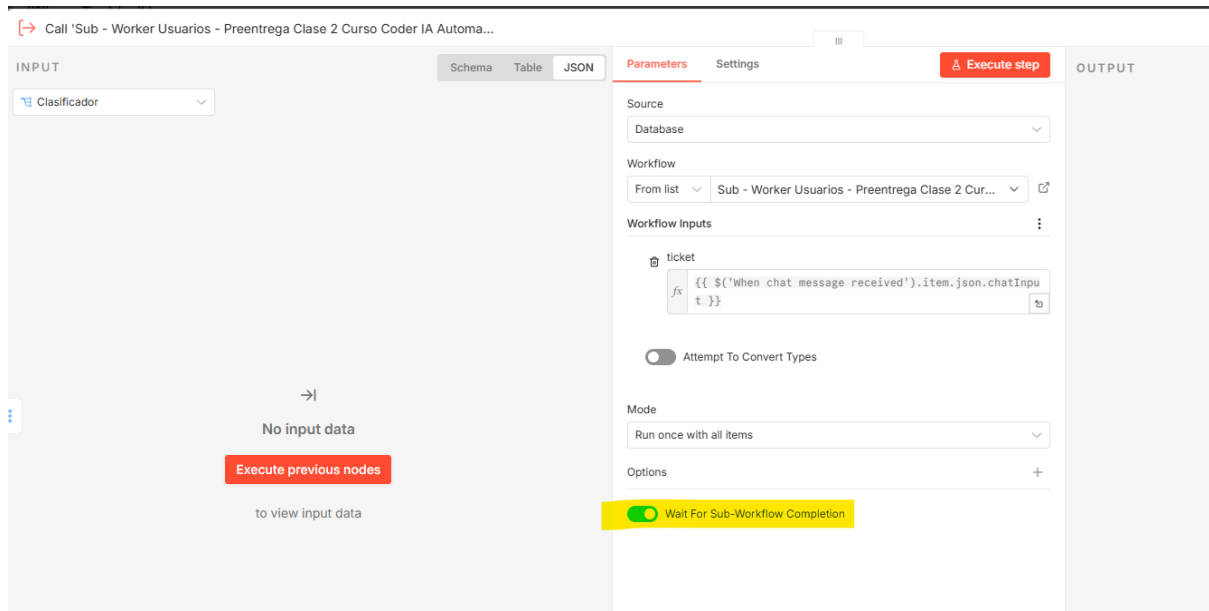
Captura — Sub-workflow Worker Infraestructura



6. Delegación y sincronización

Desde el workflow principal se utilizan nodos **Execute Sub-workflow / Call Sub-workflow** para invocar al especialista correspondiente.

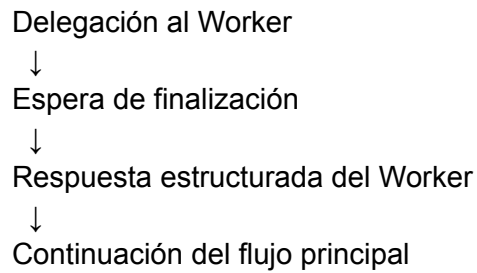
La ejecución está configurada para **esperar la finalización del Worker**, de modo que el flujo principal no continúe hasta que el especialista termine de procesar el ticket y devuelva su resultado.



Esto permite mantener una ejecución sincronizada:

Manager





7. Contrato de salida de los Workers

Los Workers devuelven una estructura JSON predecible, evitando transmitir información innecesaria al workflow principal.

Ejemplo de ejecución correcta:

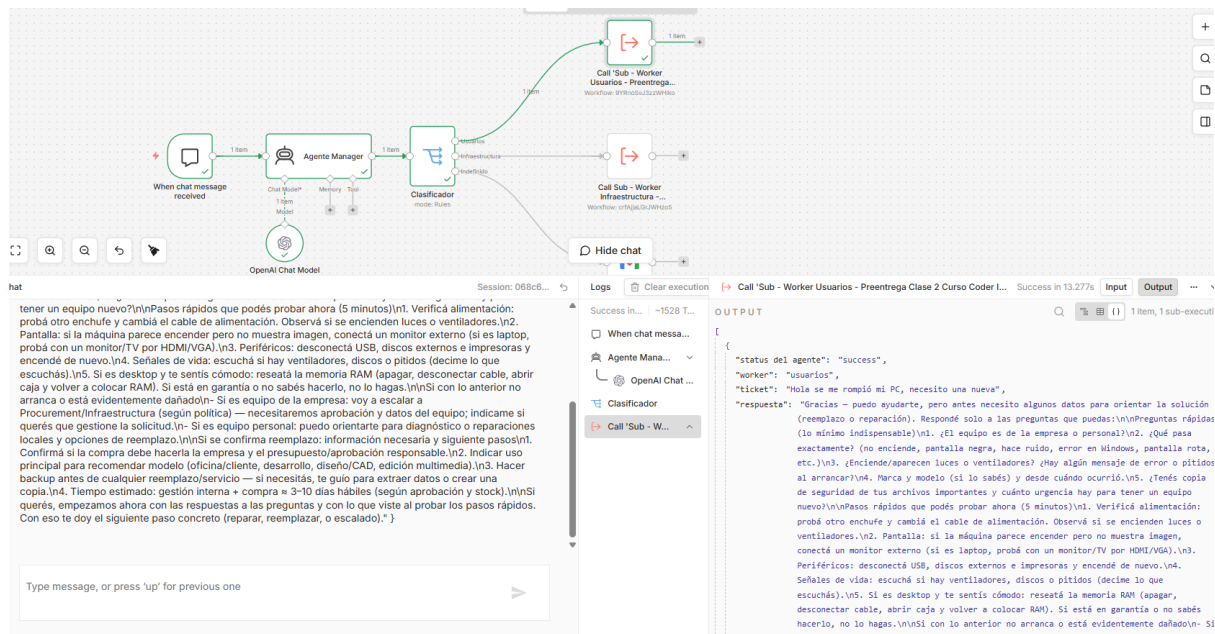
```
{  
  "status del agente": "success",  
  "worker": "infraestructura",  
  "ticket": "El servidor no responde",  
  "respuesta": "Respuesta técnica generada por el especialista"  
}
```

Para el Worker de Usuarios se utiliza la misma estructura, modificando el especialista:

```
{  
  "status del agente": "success",  
  "worker": "usuarios",  
  "ticket": "La notebook no inicia",  
  "respuesta": "Respuesta técnica generada por el especialista"  
}
```

También se contempla una salida estructurada ante errores, permitiendo mantener un contrato de datos predecible independientemente del resultado de la ejecución.

Captura — JSON de salida



8. Modularidad y separación de responsabilidades

La principal mejora respecto de una arquitectura monolítica es la separación explícita de responsabilidades.

El **Manager decide**, pero no resuelve.

El **Worker de Usuarios** resuelve únicamente problemas correspondientes a dispositivos y usuarios finales.

El **Worker de Infraestructura** se ocupa exclusivamente de redes, servidores e infraestructura.

Los casos que no pueden clasificarse de forma responsable son derivados a una persona.

Esta estructura permite que cada especialista pueda ser probado, mantenido o modificado independientemente sin alterar el resto del sistema.

9. Repositorio GitHub

Los workflows exportados en formato JSON se encuentran disponibles en el siguiente repositorio:

Repositorio:

<https://github.com/CeciliaM/Cecilia-Manzano---AI-Automation-Avanzado-Curso-Coderhouse>

Workflow principal – Manager:

<https://github.com/CeciliaM/Cecilia-Manzano---AI-Automation-Avanzado-Curso-Coderhouse---/blob/main/Preentrega%20Clase%202%20Curso%20Coder%20IA%20Automation%20avanzado%20-%20Arquitectura%20multiagente%20-%20Soporte%20t%C3%A9cnico.json>

Sub-workflow – Worker Usuarios:

<https://github.com/CeciliaM/Cecilia-Manzano---AI-Automation-Avanzado-Curso-Coderhouse---/blob/main/Sub%20-%20Worker%20Usuarios%20-%20Preentrega%20Clase%202%20Curso%20Coder%20IA%20Automation%20avanzado%20-%20Arquitectura%20multiagente.json>

Sub-workflow – Worker Infraestructura:

<https://github.com/CeciliaM/Cecilia-Manzano---AI-Automation-Avanzado-Curso-Coderhouse---/blob/main/Sub%20-%20Worker%20Infraestructura%20-%20Preentrega%20Clase%202%20Curso%20Coder%20IA%20Automation%20avanzado%20-%20Arquitectura%20multiagente.json>

10. Conclusión

La solución transforma el proceso de soporte técnico en una arquitectura multi-agente modular basada en el patrón **Manager–Worker**.

El Manager centraliza la decisión y el enrutamiento, mientras que los Workers ejecutan tareas especializadas mediante sub-workflows independientes y contratos JSON definidos. La ruta **indefinido** incorpora supervisión humana para aquellos casos donde la automatización no dispone de información suficiente para delegar el ticket de forma segura.

El resultado es una arquitectura más clara, mantenible y escalable que un único agente encargado de resolver todos los tipos de incidencias.

You have not enough Humanizer words left. Upgrade your Surfer plan.